

BST

자료구조 스터디 Week11

학생회

 Made with Gamma

INDEX

01

THEORY

02

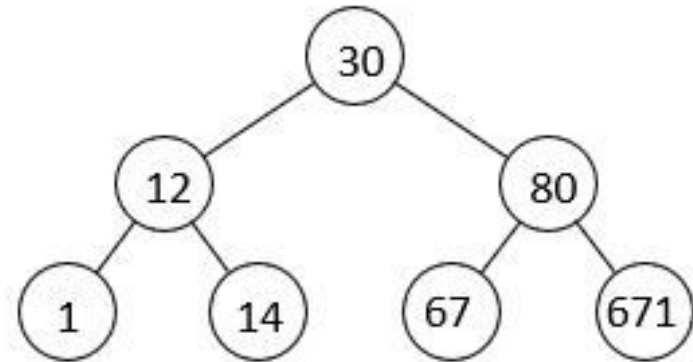
IMPLEMENTATION

03

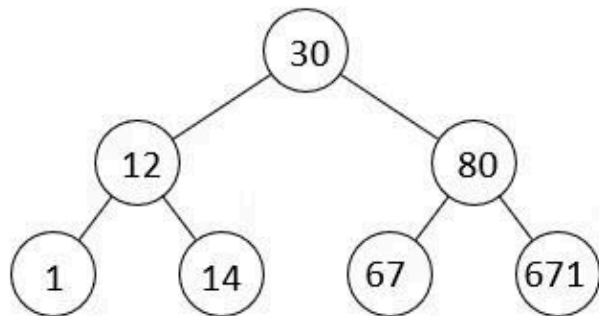
PROBLEM SOLVING

이진탐색트리

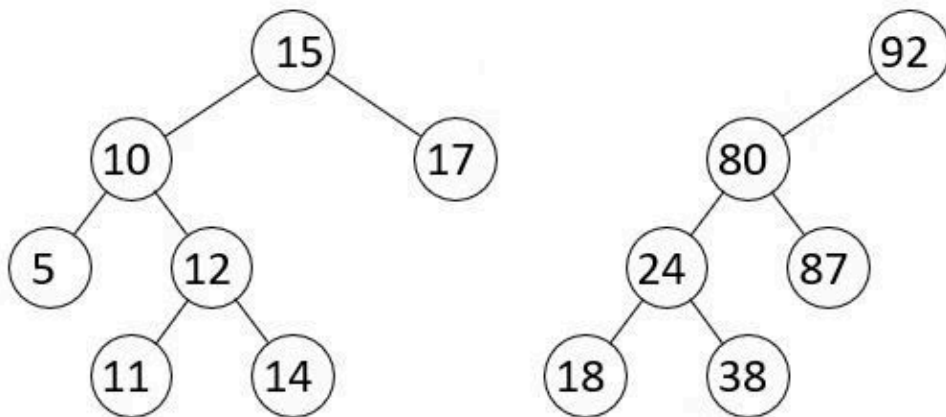
이진 트리 기반의 탐색을 위한 자료구조
각
노드의 값은 왼쪽 서브트리에 있는 어떤 노드의 값보다 크고, 오른쪽
서브트리에 있는 어떤 노드의 값보다 작다.



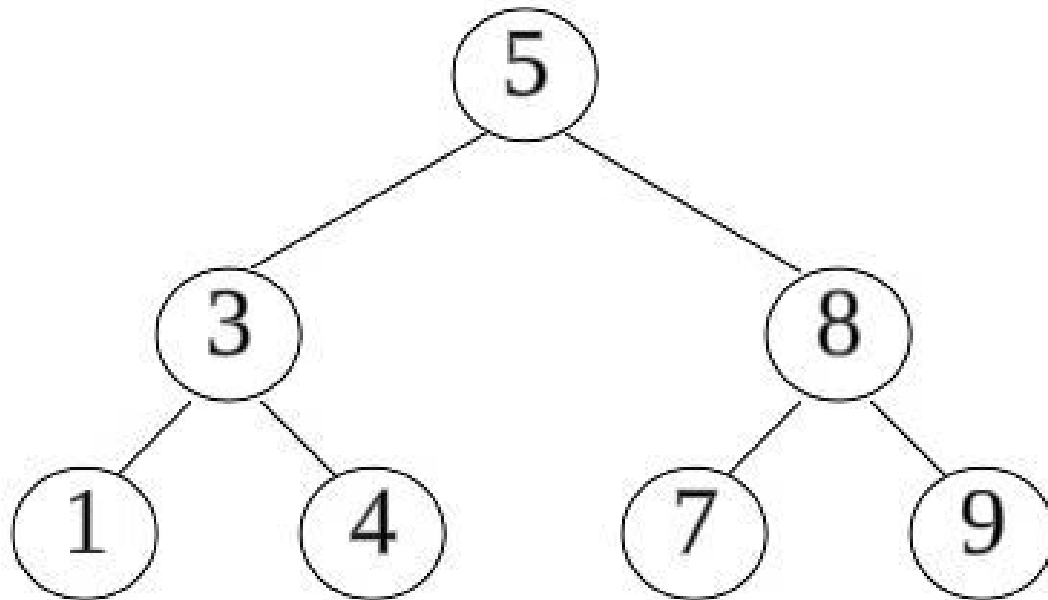
BST



구조 - 이진트리
순서 - Inorder

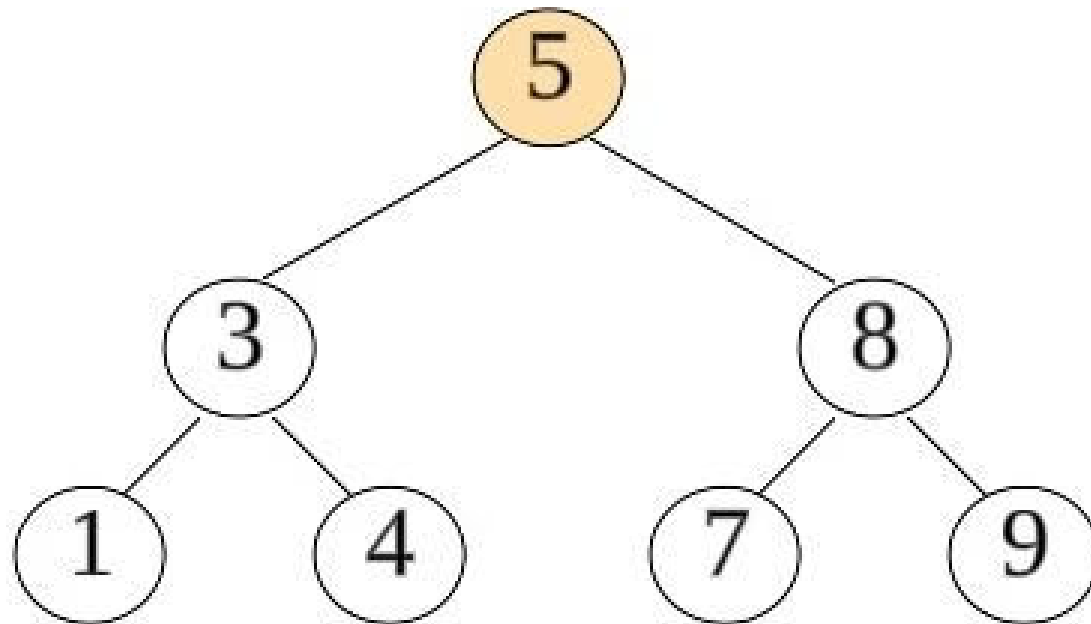


BST - Search



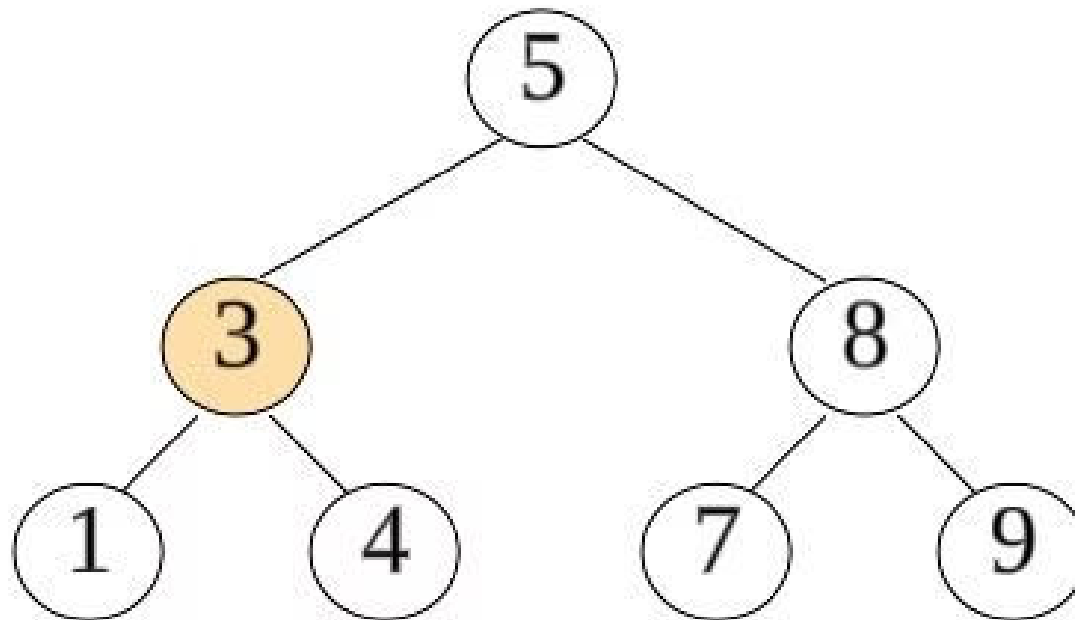
Key : 4

BST - Search



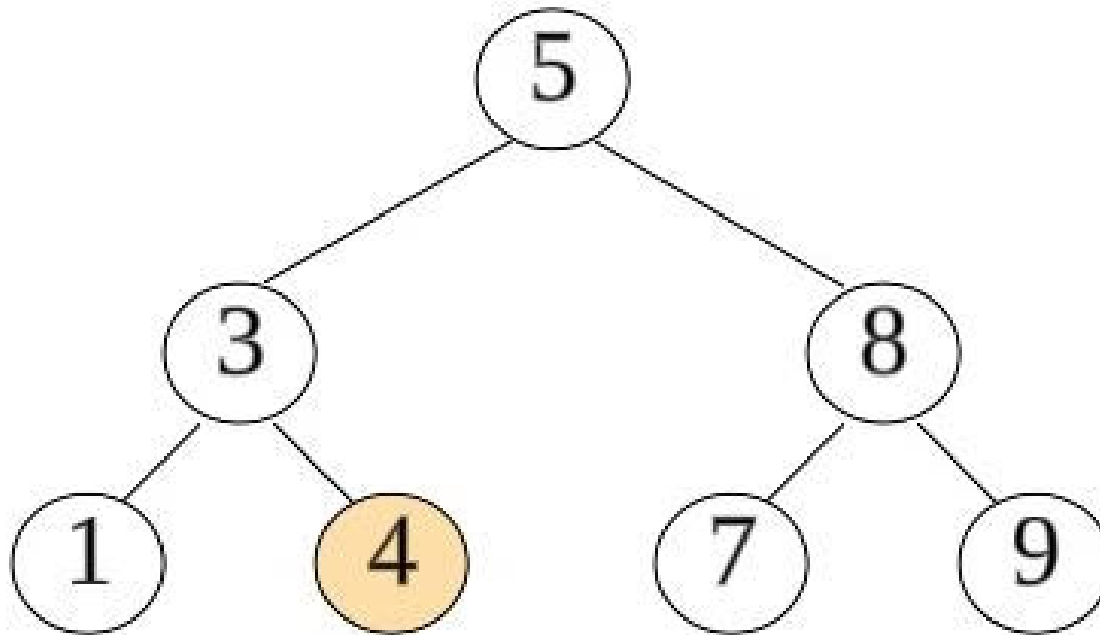
Key : 4

BST - Search



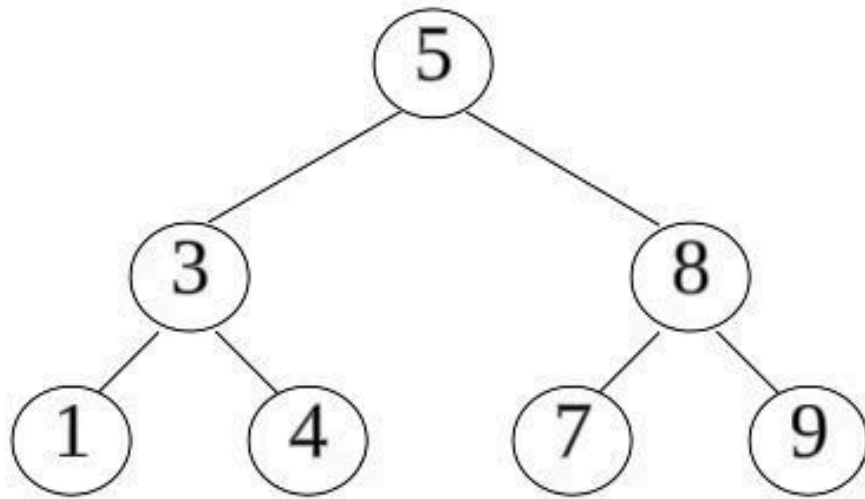
Key : 4

BST - Search



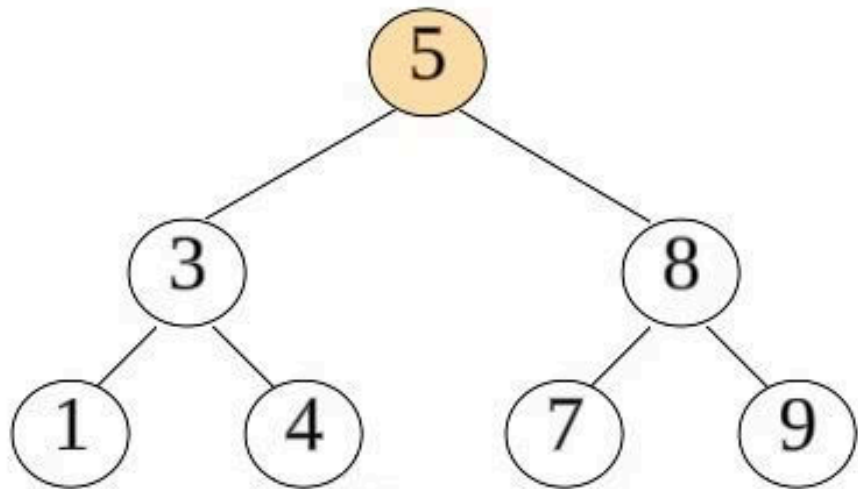
Key : 4

BST - Insert



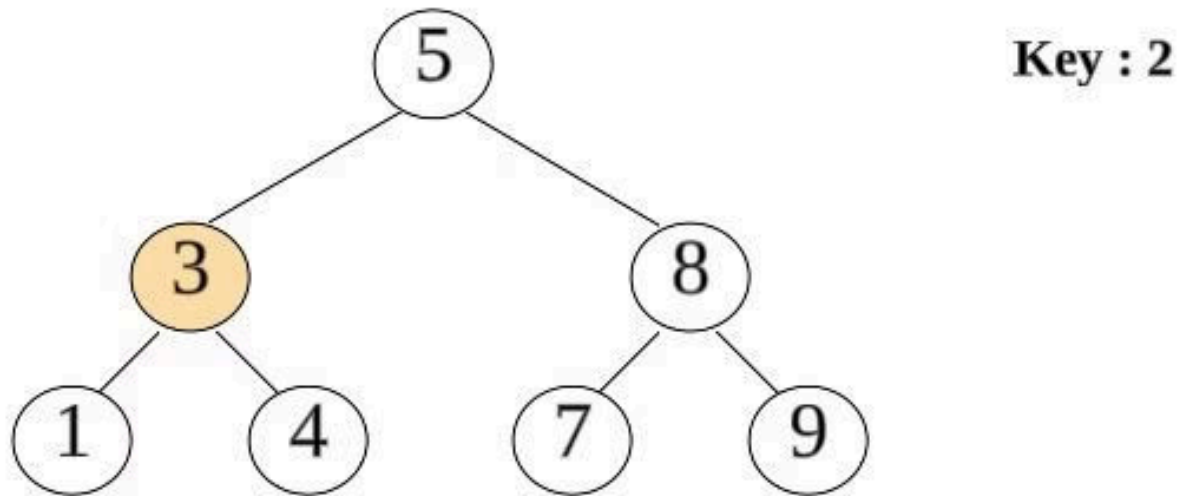
Key : 2

BST - Insert

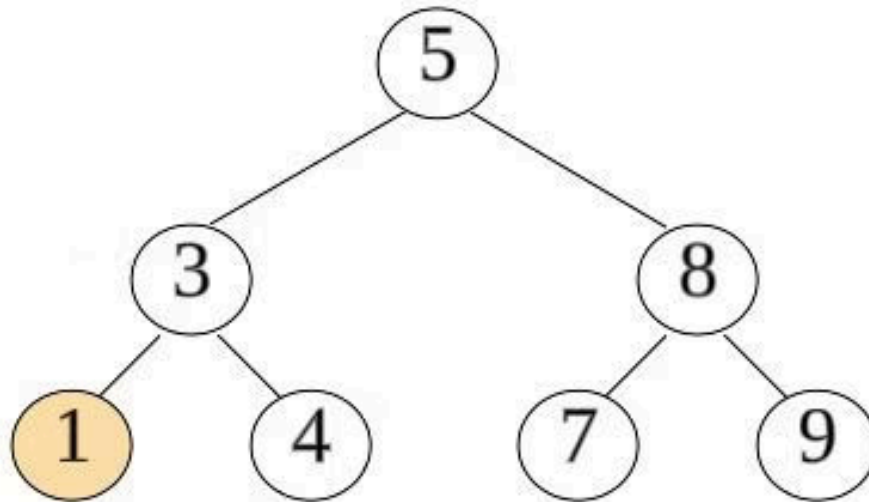


Key : 2

BST - Insert

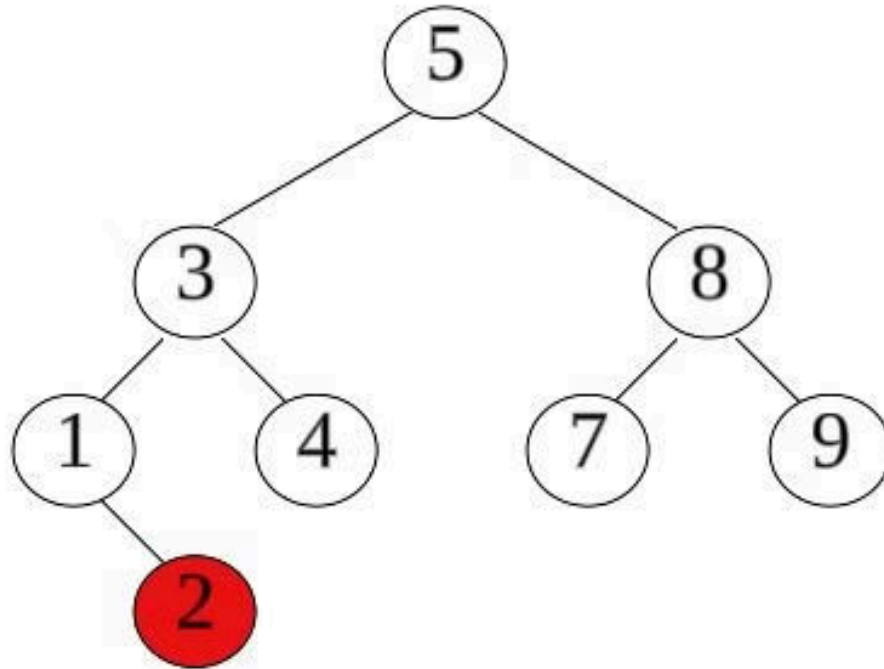


BST - Insert



Key : 2

BST - Insert



Key : 2

BST - Remove

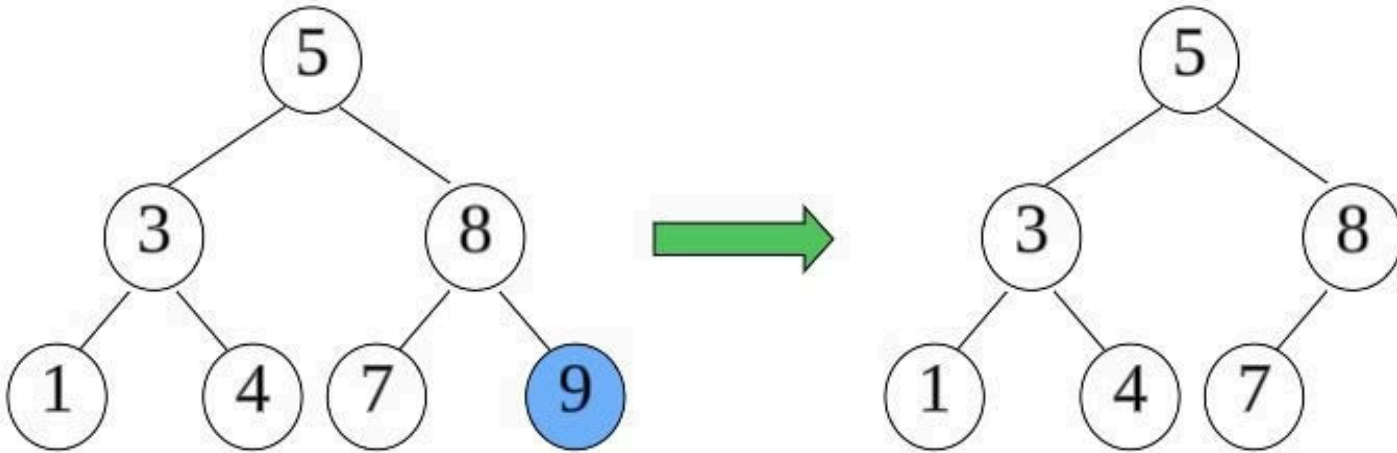
Case1) 삭제할 노드가 리프일 경우 - 그냥 삭제

Case2) 삭제할 노드의 자식이 하나인 경우 - 해당 노드의 자식과 해당 노드의 부모 연결 → 해당 노드 삭제

Case3) 삭제할 노드의 자식이 둘인 경우 - successor와 해당 노드 swap → 해당 노드 삭제 → Case1 or Case2 수행

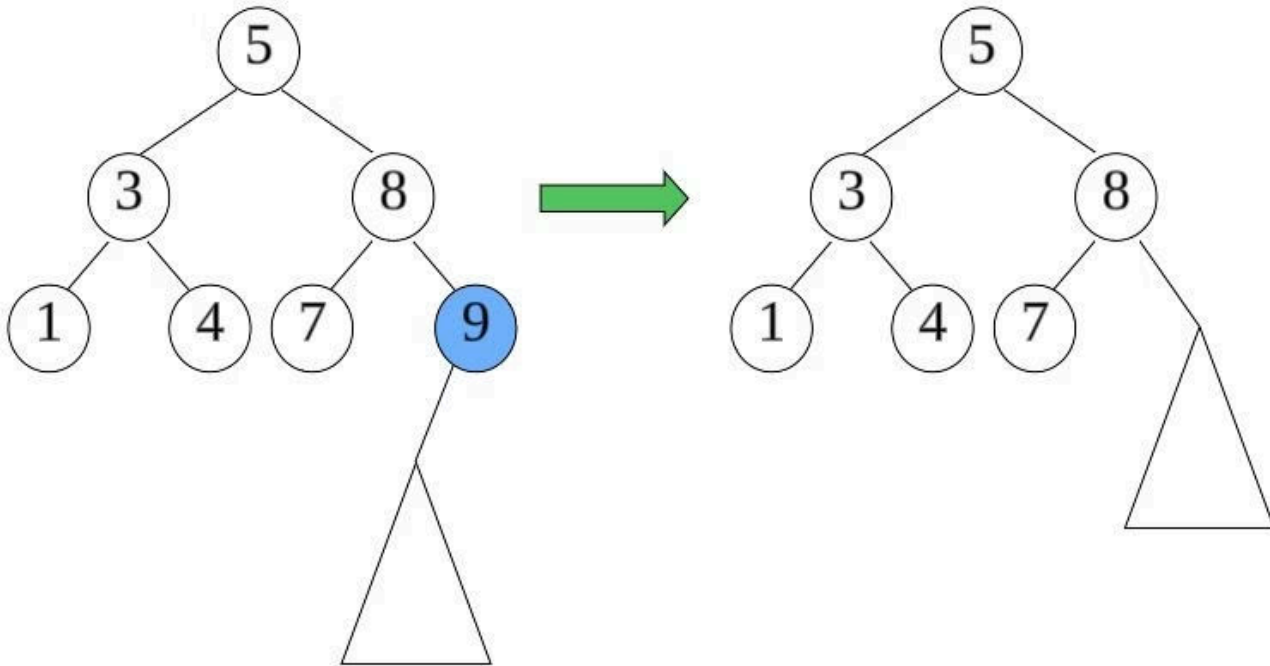
BST - Remove (Case 1)

Case1) 삭제할 노드가 리프일 경우 - 그냥 삭제



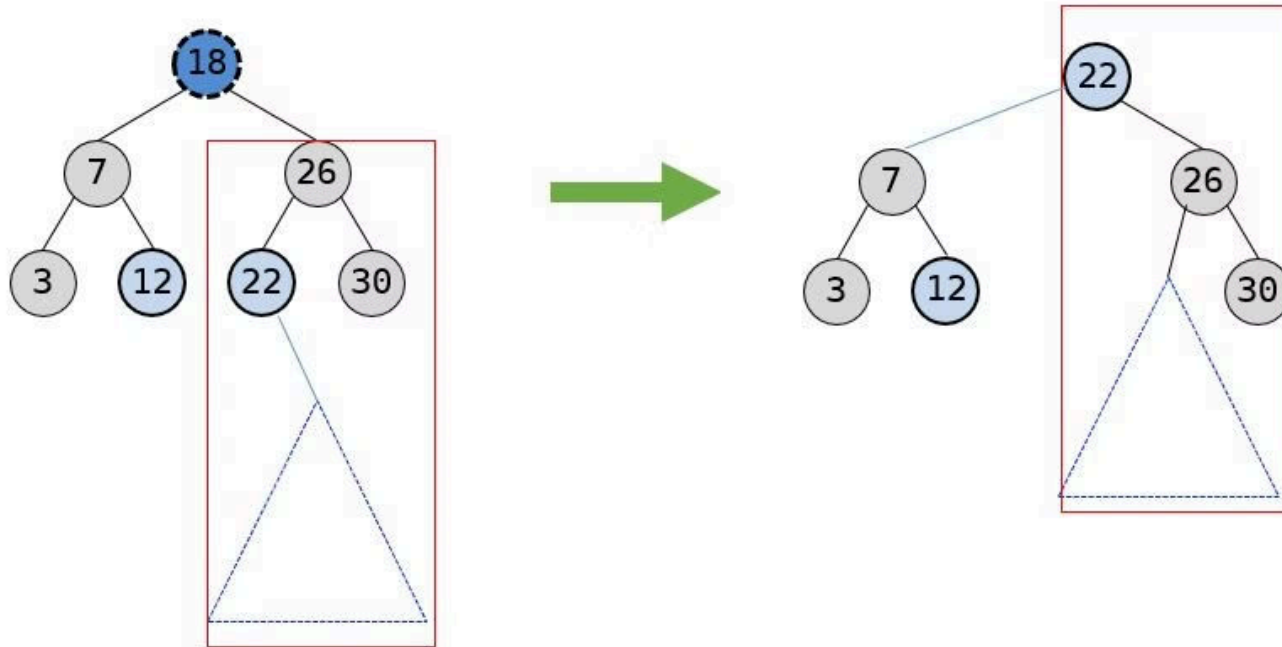
BST - Remove (Case 2)

Case2) 삭제할 노드의 자식이 하나인 경우 - 해당 노드의 자식과 해당 노드의 부모 연결 → 해당 노드 삭제



BST - Remove (Case 3)

Case3) 삭제할 노드의 자식이 둘인 경우 - successor와 해당 노드 swap → 해당 노드 삭제 → Case1 or Case2 수행



IMPLEMENTATION

우선순위 큐의 기본 연산

1 Search

탐색

2 Insert

삽입

3 Remove

삭제

노드 클래스

```
6  ✓ class Node {  
7      public:  
8          int key;  
9          Node* parent;  
10         Node* left;  
11         Node* right;  
12  
13         Node(int key) {  
14             this->key = key;  
15             parent = left = right = nullptr;  
16         }  
17     };
```

- 편의를 위해 전부 public 선언
- 값, 부모, 왼쪽 자식, 오른쪽 자식

멤버 변수 및 생성자

```
19  ✓  class BST {  
20      public:  
21          Node* root;  
22  
23          BST() {  
24              root = nullptr;  
25          }  
26
```

- BST는 트리

Search

```
27     Node* search(Node* curNode, int key) {
28         if (curNode == nullptr)
29             return nullptr;
30
31         if (curNode->key == key)
32             return curNode;
33
34         else if (curNode->key < key)
35             return search(curNode->right, key);
36
37         else
38             return search(curNode->left, key);
39     }
40
```

- 현재 노드가 nullptr라면 예외처리
- key가 현재 노드 값과 같다면 현재 노드 리턴
- key가 현재 노드 값보다 크다면 right 탐색
- key가 현재 노드 값보다 작다면 left 탐색

Insert(1)

```
41  void insert(int key) {  
42      if (search(root, key) != nullptr)  
43          return;  
44  
45      Node* newNode = new Node(key);  
46  
47      if (root == nullptr) {  
48          root = newNode;  
49          return;  
50      }  
51  
52      Node* curNode = root;  
53      Node* parNode = nullptr;
```

- New 노드가 이미 있다면 예외처리
- 만약 비어있는 BST라면 삽입 위치는 root
- 삽입할 위치 찾기 위해 curNode, parNode 선언

Insert(2)

```
55         while (curNode != nullptr) {
56             parNode = curNode;
57
58             if (curNode->key < key)
59                 curNode = curNode->right;
60
61             else
62                 curNode = curNode->left;
63         }
64
65         newNode->parent = parNode;
66
67         if (parNode->key < key)
68             parNode->right = newNode;
69
70         else
71             parNode->left = newNode;
72     }
```

- loop 종료 시 curNode는 nullptr, parNode는 리프
- 부모 지정
- 자식 지정(left, right는 조건에 맞게)

Remove(1)

```
74 void remove(int key) {  
75     Node* delNode = search(root, key);  
76  
77     if (delNode == nullptr)  
78         return;  
79  
80     Node* parNode = delNode->parent;  
81     Node* childNode;  
82  
83     if (delNode->left == nullptr && delNode->right == nullptr)  
84         childNode = nullptr;  
85  
86     else if (delNode->left == nullptr && delNode->right != nullptr)  
87         childNode = delNode->right;  
88  
89     else if (delNode->left != nullptr && delNode->right == nullptr)  
90         childNode = delNode->left;  
91
```

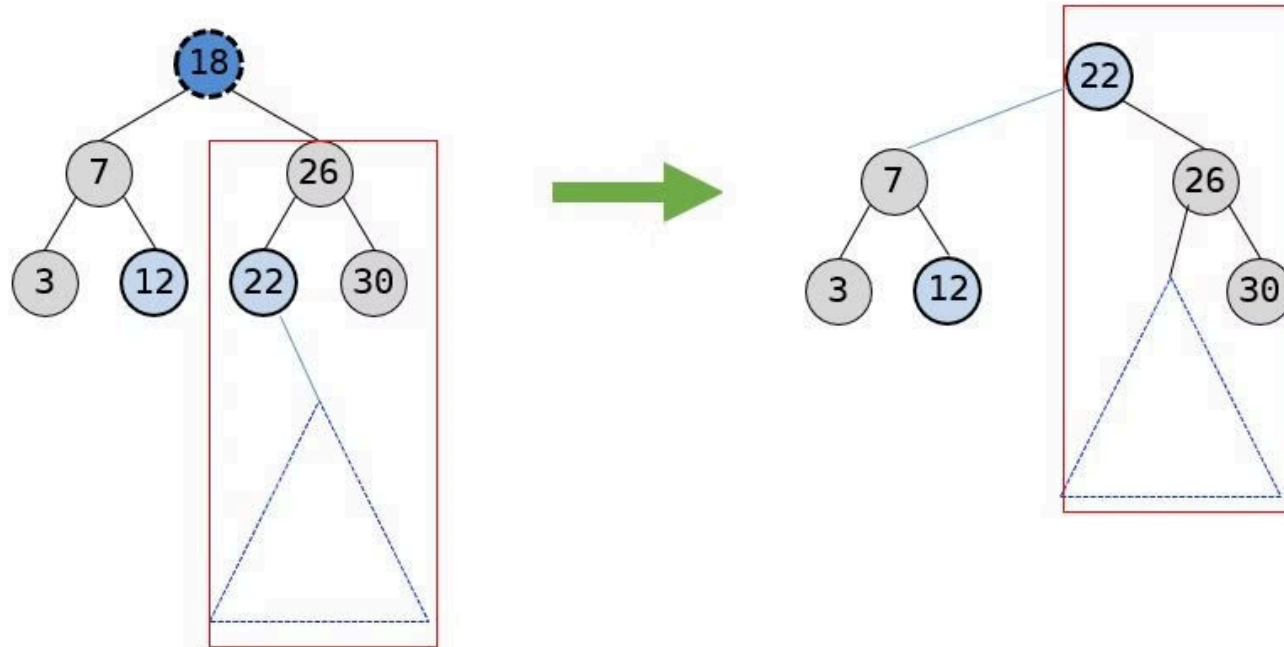
- 삭제할 노드가 없다면 예외처리
- Case1, Case2

Remove(2)

```
92         else {
93             childNode = delNode->right;
94
95             while (childNode->left != nullptr) // successor 찾기
96                 childNode = childNode->left;
97
98             delNode->key = childNode->key;
99             delNode = childNode;
100             parNode = delNode->parent;
101             childNode = delNode->right;
102         }
103
```

- Case3
- #98 - 18 → 22
- #99 ~ #101 - delNode: 18
- #101 - successor는 자식 없거나 right만 가짐!
- 이후 Case1 or Case2와 동일

BST - Remove (Case 3)



Remove(3)

```
104         if (parNode == nullptr) {
105             root = childNode;
106             root->parent = nullptr;
107         }
108
109         else if (delNode == parNode->left) {
110             parNode->left = childNode;
111
112             if (childNode != nullptr)
113                 childNode->parent = parNode;
114         }
115
116         else {
117             parNode->right = childNode;
118
119             if (childNode != nullptr)
120                 childNode->parent = parNode;
121         }
122
123         delete delNode;
124     }
125
```

- 삭제할 노드(delNode), 삭제할 노드의 부모(parNode), 삭제할 노드의 자식(childNode)
- parNode가 nullptr라면 delNode가 root
- delNode가 parNode의 left인지 right인지 확인 → childNode를 parNode에 올바르게 연결
- delNode 삭제

PROBLEM SOLVING

**THANK'S FOR
WATCHING**